

Synaptic Plasticity as Short-Term Memory: What If Your LLM Didn't Need a KV-Cache?

Interactive explainer: bdh-synaptic-memory-explainer.vercel.app (<https://bdh-synaptic-memory-explainer.vercel.app>)

Anand Tudu · September 2026 · DataForge Pathway track

TL;DR — Transformers remember by *storing*: every token's keys and values pile up in a KV-cache that grows forever. Brains remember by *changing*: synapses strengthen when neurons co-fire, then decay. Recent architectures — most prominently Pathway's Dragon Hatchling (BDH) — show the second strategy can rival the first at real model scales, with **constant memory instead of a growing cache**. This post walks through the mechanism in three equations' worth of math, and points to an interactive demo where you can break it yourself.

Every large language model you've used carries a hidden burden: the **Key-Value cache**. To generate token $n+1$, a Transformer must attend over everything it has read so far, so it stores the keys and values of every previous token. The cache grows with every step — linearly with context length, and quadratically in attention compute. For long conversations, documents, or agents that run for hours, memory becomes the bottleneck.

But your brain doesn't work that way. You are not storing a verbatim transcript of everything you've ever read. Somehow, a fixed blob of neurons — the same 86 billion you woke up with — holds a working memory of this sentence *right now*. The mechanism is old, local, and simple: **synaptic plasticity**. When two neurons fire together, the connection between them strengthens; when they stop co-firing, the trace fades. “*Neurons that fire together, wire together*” — Hebb's postulate, formalized mathematically as the outer-product update $W \leftarrow \lambda \cdot W + \eta \cdot (x \cdot x^M)$ in associative memory models [Hopfield, 1982 (<https://www.pnas.org/doi/10.1073/pnas.79.8.2554>)].

The claim this project explores — and lets you test with your own hands — is this:

Attention can be reformulated as synaptic memory: recent token interactions temporarily strengthen local connection weights via Hebbian updates, allowing a fixed-size recurrent state to process sequential context without a growing KV-cache.

The linear-attention insight: attention is a weight update

The pivotal modern result comes from Schlag, Irie & Schmidhuber (2021), who showed that linear attention's KV-state is mathematically a **fast weight program**: each token performs a rank-1 Hebbian write into a matrix, and attention is just reading that matrix back out [arXiv:2102.11174

(<https://arxiv.org/abs/2102.11174>)]. In other words, the Transformer already secretly performs a synaptic update — it just keeps the full unbounded history instead of letting it decay and interfere.

This reframing collapses the distinction between “caching” and “learning.” A KV-cache is a memory with $\lambda = 1$ (nothing fades) and $\eta = 1$ (every token writes fully, verbatim). A Hebbian state is the same memory with $\lambda < 1$ (old traces decay) and sparse writes. The difference is a policy, not a category.

The 2022–2026 wave: fixed states that compete with attention

Three recent lines of work show this isn’t just theory:

- **State-space models.** Mamba (Gu & Dao, 2023) processes million-token sequences with a fixed-size selective state — no cache at all — matching Transformer quality at practical scales [arXiv:2312.00752 (<https://arxiv.org/abs/2312.00752>)]. The LRU analysis of Orvieto et al. (2024) isolated exactly which ingredients (gating, normalization, decay) let linear recurrences close the quality gap with attention [arXiv:2303.06349 (<https://arxiv.org/abs/2303.06349>)].
- **Brain-inspired spiking graphs.** The Dragon Hatchling (BDH) from Pathway (Kosowski et al., 2025) goes furthest in the biological direction: a scale-free local graph of spiking neurons whose **synaptic state** is the working memory. BDH models at 10M–1B parameters rival GPT-2 at equal size — with no KV-cache — and individual synapses are interpretable as the memory itself [arXiv:2509.26507 (<https://arxiv.org/abs/2509.26507>), code: github.com/pathwaycom/bdh (<https://github.com/pathwaycom/bdh>)].
- **Associative-memory revival.** Modern Hopfield networks (Ramsauer et al., 2020) reframed attention itself as associative retrieval [arXiv:2006.16222 (<https://arxiv.org/abs/2006.16222>)], giving the synaptic view a clean theoretical backbone.

What you give up: interference, forgetting, and order

A fixed state is not a free lunch. Our interactive demo makes the trade-offs visceral — you drag a slider and watch them happen:

- **Interference.** Tokens with overlapping patterns write to the same synapses. Recall “the” and “a” lights up too, because they share neurons. Capacity of naive outer-product memories is $\sim 0.14 \cdot d$ patterns [Hopfield, 1982 (<https://www.pnas.org/doi/10.1073/pnas.79.8.2554>)] — for our 24-neuron demo, a handful of tokens saturates it.
- **Catastrophic forgetting.** With decay $\lambda < 1$, old traces fade multiplicatively. With $\lambda = 1$ and clamping, the matrix saturates and new memories crush old ones. Verbatim recall of an unbounded stream is simply impossible in 576 numbers.
- **Weak order information.** Outer-product traces are largely order-agnostic: “dog bites man” and “man bites dog” write nearly the same synapses. Transformers get order free from positional

embeddings plus the cache; recurrent states need extra structure.

These are precisely the failure modes the BDH paper, Mamba, and the LRU work engineer around — with sparsity, gating, and normalization respectively [arXiv:2509.26507 (https://arxiv.org/abs/2509.26507); arXiv:2312.00752 (https://arxiv.org/abs/2312.00752); arXiv:2303.06349 (https://arxiv.org/abs/2303.06349)].

The demo, in one table

Every slider in the interactive explainer (https://bdh-synaptic-memory-explainer.vercel.app) maps to a term in the update rule — and to a concept in the literature:

Demo element	Equation term	What it teaches
Decay Factor λ	the λ in $W \leftarrow \lambda \cdot W + \eta \cdot (x \cdot x^M)$	How fast memory fades — the short-term in <i>short-term memory</i>
Plasticity Rate η	the η — the write strength	Speed vs. crosstalk: fast writing also fast corrupts
Token stream	the sparse pattern x	Sparsity keeps interference low; overlap is what corrupts recall
Recall Probe	$y = W \cdot \text{cue}$	Memory is <i>content-addressed</i> : similarity, not position
Truth vs Estimate	oracle with perfect cache	The exact price paid for $O(1)$ memory, per token
Memory Budget bars	$O(1)$ vs $O(T)$	Why this matters at all: the cache is the Transformer’s scaling tax

The honest takeaway

Neither strategy wins outright. The KV-cache is exact, order-aware, and unbounded — and that last property is exactly its weakness at scale. Synaptic state is bounded, biologically plausible, and interpretable by construction — and pays for it with interference, forgetting, and weak order. The interesting question for the next few years is not *which one wins*, but **which mixtures win**: hybrid stacks that cache verbatim what matters and let the rest decay through plastic state are already appearing, and BDH’s result — matching GPT-2 quality with synaptic memory and no cache — is the strongest evidence yet that the plastic half of that mixture can carry more weight than we assumed.

Try it yourself

The interactive explainer (https://bdh-synaptic-memory-explainer.vercel.app) runs the Hebbian loop live in your browser: a 24×24 synapse matrix, a token stream, a recall probe, and an oracle that shows what a perfect cache would have recalled instead. Feed it tokens, kill λ to watch memory evaporate, crank η to watch crosstalk explode. The demo is deliberately a **toy of the mechanism, not of BDH itself** — we strip spiking thresholds and learned plasticity so the core loop is visible at a glance.

References

1. Kosowski, A., Uznański, P., Chorowski, J., Stamirowska, Z., & Bartoszkiewicz, M. (2025). *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. arXiv:2509.26507. <https://arxiv.org/abs/2509.26507>
2. Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752. <https://arxiv.org/abs/2312.00752>
3. Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Paszke, A., ... De, S. (2024). *Resurrecting Recurrent Neural Networks for Long Sequences*. ICML 2024. <https://arxiv.org/abs/2303.06349>
4. Schlag, I., Irie, K., & Schmidhuber, J. (2021). *Linear Transformers Are Secretly Fast Weight Programmers*. ICML 2021. <https://arxiv.org/abs/2102.11174>
5. Ramsauer, H., et al. (2020). *Hopfield Networks is All You Need*. arXiv:2006.16222. <https://arxiv.org/abs/2006.16222>
6. Hopfield, J. J. (1982). *Neural networks and physical systems with emergent collective computational abilities*. PNAS 79(8), 2554–2558. <https://www.pnas.org/doi/10.1073/pnas.79.8.2554>

Drafted with AI assistance (Codebuff) and reviewed, edited, and verified by the author. All claims checked against the cited papers. Blog source: `blog/bdh-synaptic-memory-vs-kv-cache.md` and `blog/bdh-synaptic-memory-vs-kv-cache.html` in the project repository (MIT license).